UNIT 4 MINI PROJECT

Infrastructure Automation using Terraform and Ansible on AWS

Submitted by: Janvi Sonawane
PRN Number: 2125UDSF1097
Subject: DevOps
Summited to : Mr Karan sir

Assignment Content: Trainer Implementation Manual

TRAINER IMPLEMENTATION MANUAL

Enterprise Smart Library Management Platform

Infrastructure Automation using Terraform and Ansible on Amazon Web Services (AWS) — Unit 4 Comprehensive Engineering Architecture Manual Curriculum

Unit Target Cloud PlatformIaC & Automation

Tools Manual Status UNIT

4: DevOps Automation

Amazon Web Services

(AWS)

Terraform (v1.7+) &

Ansible (v2.16+) v1.0.0

Production Architecture

1. Executive Summary & Infrastructure Automation Imperative

In the preceding units of the Careertiq curriculum, software engineering teams successfully transformed an enterprise business concept into an Agile product roadmap, engineered a modular Python/Flask microservice, established continuous automated testing via Pytest, and containerized the runtime using Docker and Kubernetes. However, in enterprise production, writing robust application code is only half the software lifecycle equation. Before any application can serve customer traffic, organizations must provision and maintain complex supporting infrastructure: virtual networks, subnets, routing tables, security firewalls, cloud compute instances, storage volumes, OS-level dependencies, runtime daemons, reverse proxies, and backup stores. Historically, infrastructure operations relied on manual configuration—commonly referred to in industry as "ClickOps" or manual server administration via SSH.

Manual administration introduces critical points of failure: protracted provisioning times, unrecorded human errors, security misconfigurations, and severe configuration drift between staging and production environments. Unit 4 introduces Infrastructure as Code (IaC) and Configuration Management to automate the complete infrastructure lifecycle. Using the Enterprise Smart

Library Management Platform as the reference case study, this document demonstrates how modern software organizations declare cloud infrastructure via Terraform and automate server configuration via Ansible on AWS.

📌 CORE ARCHITECTURAL OBJECTIVE

Eliminate manual server setup through declarative Infrastructure as Code (T erraform) and idempotent Configuration Management (Ansible). By codifying the infrastructure lifecycle, organizations achieve 100% environment reproducibility, zero configuration drift, immutable audit trails, and automated zero-touch server provisioning.

Figure 1.1: Evolution of Software Infrastructure Management from Manual Runbooks to Cloud-Native IaC

2. Comprehensive Infrastructure Requirements Analysis

Infrastructure planning must always precede provisioning. T o ensure optimal performance, security isolation, and cost governance, engineers must analyze the exact computational and networking prerequisites of the application components.

2.1 Application Components vs. Cloud Infrastructure Mapping

The Enterprise Smart Library Management Platform consists of distinct functional layers. T able 2.1 maps each application layer to its required AWS infrastructure primitive.

Application
Component Runtime Function Target AWS
Infrastructure Operational Specifications
Python Web APIFlask 3.0 REST Endpoints,
Gunicorn
WSGI Server
AWS EC2 Instance
(Compute)
Instance type: t2.micro (1 vCPU, 1
GiB RAM), Ubuntu

22.04 LTS, EBS 20GB gp3 root

volume.
Database
Storage
Library book records,
borrower accounts, loan
transactions
Persistent Storage / Local
DB
Thread-safe SQLite store with
mounted volume persistence;
exportable to AWS RDS
PostgreSQL for scale.
Web Reverse
Proxy
SSL T ermination, HTTP
request buffering, static
proxy
Nginx Web Server Listens on public Port 80; reverse
proxies internal
requests to Gunicorn Docker
container on Port 5000.
Container
Runtime
Executes packaged Docker
container image from Unit
3
Docker Engine &
containerd
Container restart policy always;
network port mapping
5000:5000; isolated cgroups &
namespaces.
Virtual NetworkNetwork boundary, CIDR

isolation, packet routing
AWS Virtual Private
Cloud (VPC)
Dedicated VPC 10.0.0.0/16;
Public Subnet 10.0.1.0/24; Internet
Gateway (IGW); Route T able.
Network
Security
Ingress traffic filtering,
port isolation
AWS Security Group
(Stateful)
Restricted Ingress: Port 22 (SSH
Management), Port 80 (HTTP),
Port 5000 (Direct API).
Unrestricted Egress.
Backup StorageNightly database dumps, audit
logs, T erraform
state
AWS S3 Object BucketPrivate encrypted bucket
(AES-256); S3 Versioning
enabled; lifecycle policies for
disaster recovery.

2.2 Multi-Tier Environment Variations

Infrastructure requirements differ across lifecycle environments. The automation architecture must accommodate:
Development Environment: Optimized for rapid developer feedback and minimal cost. Utilizes single Availability Zone (AZ) deployment, ephemeral storage, and relaxed security groups for local testing.
Testing & Staging Environment: 100% replica of production. Executes automated CI/CD integration tests, load tests, and security vulnerability scans.
Production Environment: High availability, multi-AZ resilience, Elastic IP binding, encrypted S3 backups, strict least-privilege IAM roles, and automated alerting.

3. Cloud Architecture Design on Amazon Web Services (AWS)

Modern cloud architectures separate public traffic routing from internal application runtime. Figure 3.1 illustrates the complete enterprise cloud deployment topology designed for the Library Management Platform.

•
•
•

Figure 3.1: Enterprise AWS Cloud Architecture Topology for Library Management Platform

3.1 Architectural Layer Breakdown

Network Isolation Layer: The platform resides within a custom AWS Virtual Private Cloud (VPC) assigned the CIDR block 10.0.0.0/16. An Internet Gateway (IGW) is attached to the VPC to enable bidirectional internet communication. A Public Subnet (10.0.1.0/24) hosts the compute instance, with default routes (0.0.0.0/0) directed to the IGW.

Security & Ingress Layer: A stateful AWS Security Group acts as a virtual firewall. Ingress traffic is restricted to Port 22 (SSH administrator access), Port 80 (Nginx HTTP web traffic), and Port 5000 (direct application access). Outbound egress is open to allow APT updates and Docker image pulls. Compute & Runtime Layer: An AWS EC2 instance running Ubuntu 22.04 LTS executes Docker Engine. Inside Docker, the Enterprise Smart Library container runs isolated, exposing port 5000.

Nginx functions as a high-performance reverse proxy on Port 80, buffering client requests and forwarding traffic to the container.

Persistent Storage & State Layer: An AWS S3 bucket maintains versioned, encrypted backups of library transactions. Concurrently, S3 and AWS DynamoDB provide remote state storage and distributed locking for T erraform execution.

4. Infrastructure as Code (IaC) with HashiCorp Terraform

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable definition files rather than physical hardware configuration or interactive GUI tools. HashiCorp T erraform is the industry-standard cloud-agnostic IaC engine.

-
-
-
-

Figure 4.1: Terraform Declarative Execution Pipeline and State Reconciliation Engine

4.1 Core Concepts and Principles of Terraform

Declarative Syntax (HCL): HashiCorp Configuration Language allows engineers to define the desired end state of infrastructure. The engineer specifies what resources should exist, and T erraform calculates how to achieve that state.

Terraform Providers: Plugins that interact with cloud platforms via official APIs. For AWS, the hashicorp/aws provider translates HCL declarations into AWS REST API requests.

State Management (terraform.tfstate): T erraform records resource metadata, dependencies, and IDs in a state file. This state file serves as the single source of truth mapping declared resources to real-world cloud objects.

State Locking & Concurrency: In team environments, concurrent T erraform executions can corrupt the state file. T erraform integrates with AWS DynamoDB to enforce distributed mutex locking during operations.

4.2 The Five-Stage Terraform Command Lifecycle

Terraform Command	Internal Execution Mechanism	Operational Outcome
terraform init	Initializes the working directory, downloads required provider plugins (hashicorp/aws), and connects to remote state backends.	Creates .terraform/ directory and locks dependencies in .terraform.lock.hcl.
terraform validate	Parses configuration files syntactically and semantically without making remote API calls.	Confirms HCL syntax correctness, valid variable types, and attribute presence.
terraform plan	Queries cloud APIs to inspect current state, compares it with local HCL declarations, and constructs an execution graph.	Generates an execution plan detailing resources to be created (+), modified (~), or destroyed (-).
terraform apply	Executes the dependency graph in parallel, calling AWS APIs to provision resources in topological order.	Provisions VPC, Subnet, Security Group, S3, and EC2, then updates terraform.tfstate.
terraform destroy	Reverses the dependency graph and systematically deprovisions all managed cloud resources.	T erminates EC2, deletes S3 buckets, removes VPC, and frees cloud

infrastructure costs.

-
-
-
-

5. Configuration Management with Ansible

While T erraform provisions cloud fabric (hardware, networking, instances), it is not designed for granular operating system and software configuration. That task belongs to Configuration Management tools such as Ansible.

Figure 5.1: Ansible Agentless Control Node Architecture & Modular Role Execution

5.1 Why Ansible? Key Architectural Advantages

Agentless Architecture: Unlike legacy tools (Puppet, Chef) that require proprietary daemons running on target servers, Ansible operates over standard OpenSSH. T arget servers require only a Python interpreter.

Push-Based Model: The Ansible Control Node pushes configurations directly to target hosts on demand, eliminating client pull delays and central server load.

Idempotency: An operation is idempotent if executing it multiple times produces the exact same outcome without unintended side effects. If a package (e.g., Docker) is already installed, Ansible reports ok and skips reinstallation.

Human-Readable YAML: Playbooks and roles are declared in straightforward YAML syntax, facilitating code reviews and cross-team collaboration.

5.2 Modular Ansible Roles Hierarchy

Enterprise configurations are structured into reusable Roles. For the Smart Library Platform: roles/common: Updates APT repository caches, installs system utilities (curl, git, jq), and configures UFW firewall.

roles/docker: Adds official Docker GPG keys, installs Docker CE and containerd, enables systemd daemons, and adds non-root users to the docker security group.

roles/nginx: Installs Nginx, generates dynamic reverse proxy configurations via Jinja2 templates, and proxies Port 80 to container Port 5000.

roles/library_app: Deploys the containerized application artifact from Unit 3, mounts persistent database directories, launches the container with restart policies, and verifies health endpoints.

6. The Two-Tier DevOps Orchestration Handshake

DevOps excellence requires combining T erraform and Ansible into a unified, automated delivery continuum. Figure 6.1 illustrates the two-tier orchestration handshake.

-
-
-
-
-
-
-
-

Figure 6.1: Two-Tier DevOps Orchestration Handshake: Terraform Provisions, Ansible Configures

6.1 The Orchestration Handoff Mechanism

The central engineering challenge is connecting the dynamic outputs of T erraform (such as randomly assigned AWS public IP addresses) to Ansible's static inventory. Modern DevOps solves this via automated templating:

Automated Handshake Sequence:

1. T erraform provisions EC2 instance and outputs: app_server_public_ip = "54.210.88.14"

2. CI pipeline or wrapper script queries: terraform output -raw app_server_public_ip

3. Output dynamically populates Ansible inventory:

[library_servers]

library-host ansible_host=54.210.88.14 ansible_user=ubuntu

4. Ansible executes playbook against newly created host via SSH.

7. Environment Parity & Multi-Tier Promotion Pipeline

Environment inconsistency is a primary driver of software deployment failure. If development, testing, and production servers differ in OS patch levels, Python minor versions, or firewall configurations, software that passed tests will fail in production.

Figure 7.1: Environment Parity and Promotion Pipeline across Development, Staging, and Production

7.1 Achieving 100% Parity via Code

Terraform Workspaces: A single T erraform codebase defines infrastructure across environments by switching workspaces: terraform workspace select dev vs. prod. Variables dynamically adjust instance types and replica counts while maintaining identical architecture.

Ansible Group Variables: Environment-specific parameters (e.g., database connection strings, logging verbosity) reside in group_vars/dev.yml and group_vars/prod.yml, while tasks remain completely identical.

8. Infrastructure Validation & Security Hardening

Deploying infrastructure is incomplete without rigorous post-deployment validation. Production systems mandate automated smoke testing across all architectural layers.

Figure 8.1: Four-Tier Post-Deployment Infrastructure and Application Validation Gate

8.1 The Four Validation Gates

Validation GateVerification Objective Automated Validation Command Gate 1: Network & OS Confirms instance reachable via network, SSH port 22 open, and Ubuntu kernel initialized. nc -zv -w 5 $EC2_IP 22 and ssh -i key.pem ubuntu@$EC2_IP uptime Gate 2: Daemons & Ports Verifies system services active, Docker daemon running, and Nginx listening on Port 80. ansible -m systemd -a "name=docker state=started" library_servers Gate 3: App Container Ensures Docker container status is Up, internal port 5000 mapped, SQLite database writable. ssh ubuntu@$EC2_IP "docker ps --filter name=library-app --format '{{.Status}}'" Gate 4: API Smoke Tests Executes HTTP GET and POST requests against REST endpoints to confirm live application health. curl -s -f http://$EC2_IP/api/health | jq .status (Must return "UP")

•

•

9. Student Project Adaptation: Student Record & Academic

Platform

Following the trainer's demonstration on the Library Management Platform, students independently automate the cloud infrastructure for their parallel academic project: the Enterprise Student Record and Academic Management Platform. Curriculum

DimensionTrainer Case Study (Library Platform)

Student Implementation (Academic Platform)

Application DomainLibrary book circulation, borrowings, inventory

Student records, course enrollment, academic marks

Compute Sizing AWS EC2 t2.micro (1 vCPU, 1GB RAM)AWS EC2 t2.micro (1 vCPU, 1GB RAM) Terraform

VPC & SGVPC: 10.0.0.0/16, SG: 22, 80, 5000 VPC: 10.1.0.0/16, SG: 22, 80, 8080

Ansible Role Focus Docker container running Library Flask app

Docker container running Student Record Flask app

API Smoke Tests GET /api/health, GET /api/books, POST /api/borrow

GET /api/health, GET /api/students, POST /api/marks

9.1 Evaluation Rubric & Submission Deliverables

1. Terraform Configuration Files (30%): Clean, modular HCL definitions for VPC, Subnet,

Security Group, EC2, and S3. Must execute cleanly without manual intervention.

2. Ansible Automation Roles (30%): Modular roles for system setup, Docker installation, Nginx

configuration, and application deployment. Playbooks must demonstrate strict idempotency.

3. End-to-End Cloud Deployment (20%): Successful deployment of the containerized

application on AWS EC2, reachable via public IP on Port 80.

4. Validation & Technical Report (20%): Submission of automated smoke test logs, architecture

diagrams, and GitHub repository link.

.

Hands-on Lab Guide: Prerequisites and AWS

Setup HANDS-ON LAB GUIDE: CHAPTER 2

Chapter 2: Prerequisites & AWS Environment Setup

2.1 Installing AWS CLI, Terraform, and Ansible — Enterprise Smart Library

Management Platform

Curriculum Module Target Platform Focus Tools Release Status Unit 4:

Infrastructure

Automation

Ubuntu 22.04 LTS /

macOS / WSL2

AWS CLI v2, Terraform,

Ansible

v1.0.0 Verified Lab

Guide

1. Chapter 2 Overview & Objectives

Before executing Infrastructure as Code (IaC) or Configuration Management automation, engineers must establish a standardized, deterministic local workstation environment. This guide provides exact, production-verified instructions to install, configure, authenticate, and validate the three foundational tools required for the Enterprise Smart Library Management Platform:

AWS Command Line Interface (AWS CLI v2): The unified tool to manage AWS cloud services, query resources, and authenticate programmatic API sessions.

HashiCorp Terraform (v1.7+): The declarative Infrastructure as Code engine used to provision cloud fabric (VPCs, Subnets, Security Groups, EC2 compute, and S3 storage).

Ansible Core (v2.16+): The agentless configuration management engine executing playbooks over OpenSSH to prepare servers, install Docker and Nginx, and launch the application.

📌 LAB PREREQUISITES & SYSTEM REQUIREMENTS

Operating System: Linux (Ubuntu 20.04/22.04 LTS recommended), Debian 11/12, macOS (Apple Silicon or Intel), or Windows 10/11 running Windows Subsystem for Linux 2 (WSL2 Ubuntu).

Hardware: Minimum 2 CPU cores, 4 GB RAM, 15 GB free disk space, and outbound broadband internet connectivity on ports 80, 443, and 22.

2. Section 2.1: Installing AWS CLI, Terraform, and Ansible

2.1.1 Step 1: Workstation Baseline Utilities

Ensure local package repositories are refreshed and necessary archive utilities, compilers, and security certificates are installed:

💻 Terminal: Updating Repositories and Installing Baseline Tools

```
# Update local package indexes and upgrade outdated packages
```

```
sudo apt-get update && sudo apt-get upgrade -y
# Install core system utilities, cURL, unzipping tools, and Python prerequisites
sudo apt-get install -y curl wget unzip jq git software-properties-common ca-certificates gnupg apt-
```
transport-https python3 python3-pip python3-venv

2.1.2 Step 2: Installing AWS Command Line Interface (AWS CLI v2)

AWS CLI version 2 is the official modern binary distribution for managing Amazon Web Services. Unlike version 1, AWS CLI v2 is distributed as a self-contained bundle that does not rely on host Python environments.

💻 Terminal: Downloading and Installing AWS CLI v2 on Linux (x86_64)

Careertiq Logo

-
-
-

```
# 1. Download official AWS CLI v2 installer package
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" # 2. Extract the
downloaded archive
unzip -q awscliv2.zip
# 3. Execute installation script (creates binaries in /usr/local/bin) sudo ./aws/install --update
# 4. Clean up installation artifacts rm -rf aws
awscliv2.zip
# 5. Verify installation aws --
version
```

💡 Note for macOS & Windows WSL2 Users

macOS (Homebrew): Run brew install awscli terraform ansible.

macOS (Official PKG): Download AWSCLIV2.pkg from Amazon's official documentation.

Windows WSL2: Follow the Linux terminal commands above inside your Ubuntu WSL2 bash shell.

2.1.3 Step 3: Installing HashiCorp Terraform CLI

HashiCorp signs all package releases using official GPG keys. We securely register HashiCorp's official APT repository and install the verified T erraform binary:

💻 Terminal: Registering HashiCorp GPG Key and Installing Terraform

```
# 1. Add HashiCorp's official GPG signing key to secure keyrings
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-
archive-keyring.gpg
# 2. Add HashiCorp official Debian/Ubuntu repository source list
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
# 3. Update APT package index to index HashiCorp repository sudo apt-get update
# 4. Install T erraform CLI
sudo apt-get install -y terraform
# 5. Enable optional shell autocomplete for Bash / Zsh terraform -install-
autocomplete
# 6. Verify T erraform installation terraform -version
```

2.1.4 Step 4: Installing Ansible Core & Dependencies

Ansible requires Python 3.10+ on the control node. On Ubuntu, installing from the official Ansible Personal Package Archive (PPA) ensures access to the latest stable release of Ansible Core and built-in modules:

💻 Terminal: Adding Ansible PPA and Installing Ansible Core

```
# 1. Add Ansible official PPA repository
sudo add-apt-repository --yes --update ppa:ansible/ansible # 2. Install Ansible
Core package
sudo apt-get install -y ansible
# 3. Install community and AWS cloud collection extensions
```
ansible-galaxy collection install amazon.aws community.general
```
# 4. Verify Ansible installation and Python runtime path ansible --version
```

3. Section 2.2: AWS Programmatic Authentication & STS Verification

T o permit T erraform and AWS CLI to provision infrastructure, you must configure programmatic credentials using an IAM user with appropriate permissions (e.g., EC2, VPC, S3, and Security Group policies).

3.1 Configuring AWS Credentials via CLI

💻 Terminal: Running 'aws configure' Interactive Setup

```
$ aws configure
```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE

AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY Default region name [None]: us-east-1

Default output format [None]: json

3.2 Verifying AWS Security Token Service (STS)

Verify that your credentials are functioning and identify the authenticated IAM principal:

💻 Terminal: Verifying Authenticated IAM Identity

```
aws sts get-caller-identity
```
📄 Expected Terminal Output: JSON Caller Identity

```
{
"UserId": "AIDAS6SAMPLEUSEREXAMPLE",
"Account": "123456789012",
"Arn": "arn:aws:iam::123456789012:user/devops-trainer-admin"
}
```
⚠ SECURITY GOVERNANCE NOTE

Never hardcode AWS Access Keys into Git repositories or T erraform `.tf` files. The AWS CLI stores credentials securely in `~/.aws/credentials` and `~/.aws/config`, which T erraform automatically detects and uses at runtime.

4. Section 2.3: SSH Key Pair Generation & EC2 Import

Ansible operates agentlessly over OpenSSH. T o enable Ansible to authenticate securely with the provisioned EC2 instance, generate a 4096-bit RSA key pair on your local machine and import the public key directly to AWS:

4.1 Generating 4096-Bit RSA SSH Key Pair

💻 Terminal: Generating Key Pair with Strict File Permissions

```
# 1. Create secure SSH directory if not present
mkdir -p ~/.ssh

# 2. Generate a 4096-bit RSA key without passphrase for automated CI/CD execution
ssh-keygen -t rsa -b 4096 -f ~/.ssh/library-aws-key -N "" -C "devops@careertiq-library"

# 3. Restrict private key permissions to read-only for owner (Mandatory for OpenSSH)
chmod 400 ~/.ssh/library-aws-key

chmod 644 ~/.ssh/library-aws-key.pub

# 4. Confirm key generation
ls -la ~/.ssh/library-aws-key*
```

4.2 Importing Public Key into AWS EC2 Key Pairs

💻 Terminal: Registering Public Key in Region us-east-1

```
aws ec2 import-key-pair --key-name "library-aws-key" --public-key-material fileb://~/.ssh/library-aws-key.pub --region us-east-1
```

📄 Expected Output: AWS EC2 Key Pair Confirmation

```
{
"KeyFingerprint": "b4:37:38:12:44:98:ef:5a:7b:19:30:fa:8c:11:7d:e2", "KeyName":
"library-aws-key",
"KeyPairId": "key-0a9b8c7d6e5f41234"
}
```

5. Section 2.4: Toolchain Verification Matrix & Sanity Checks

Execute the following verification checklist to ensure all tools are functioning correctly before proceeding to T
erraform HCL provisioning:

Tool Name Verification CommandExpected Version / OutputStatus AWS
CLI v2 aws --version aws-cli/2.15.x or higher VERIFIED AWS Auth
aws sts get-caller-
identity
Valid AWS Account ID & IAM User
ARN
VERIFIED
Terraform CLI terraform -version Terraform v1.7.x or higherVERIFIED Ansible
Core ansible --version ansible [core 2.16.x or
higher]
VERIFIED
SSH Key Pair aws ec2 describe-key-
pairs
library-aws-key present in AWSVERIFIED

6. Troubleshooting Common Environment Setup Errors

Symptom / Error Message	Root Cause	Remediation Step
aws: command not found	Binary directory /usr/local/bin is missing from current user PATH environment variable.	Run export PATH=$PATH:/usr/local/bin and append to ~/.bashrc or ~/.zshrc.
Signature verification failed (GPG Error)	System CA certificates are outdated or firewall is intercepting HashiCorp GPG key downloads.	Run sudo apt-get install --reinstall ca-certificates and re-fetch key via official URL.
Permissions 0644 for private key are too open	Private SSH key has permissive permissions allowing other users to read it.	OpenSSH refuses connection. Run chmod 400 ~/.ssh/library-aws-key to restrict access exclusively to the file owner.
InvalidAccessKeyId: The AWS Access Key Id does not exist	T ypo in Access Key or IAM user was deleted/deactivated in AWS Management Console.	Generate a fresh Access Key pair in the AWS IAM Console and re-run aws configure.

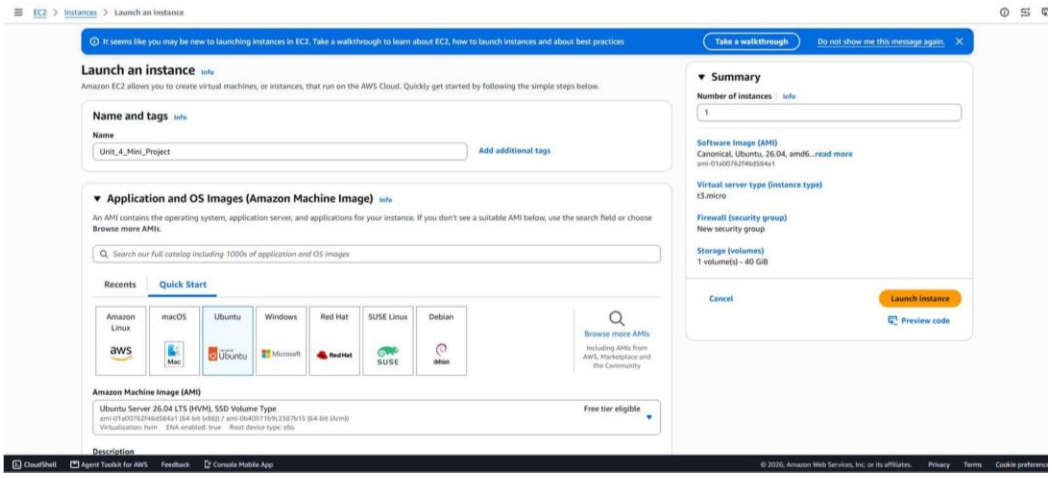
✅ NEXT STEP: INFRASTRUCTURE AS CODE PROVISIONING

With Chapter 2 setup complete, your workstation is ready to proceed to Chapter 3: Terraform Infrastructure as Code Suite to provision the AWS VPC, Security Group, EC2 compute host, and S3 backup bucket.

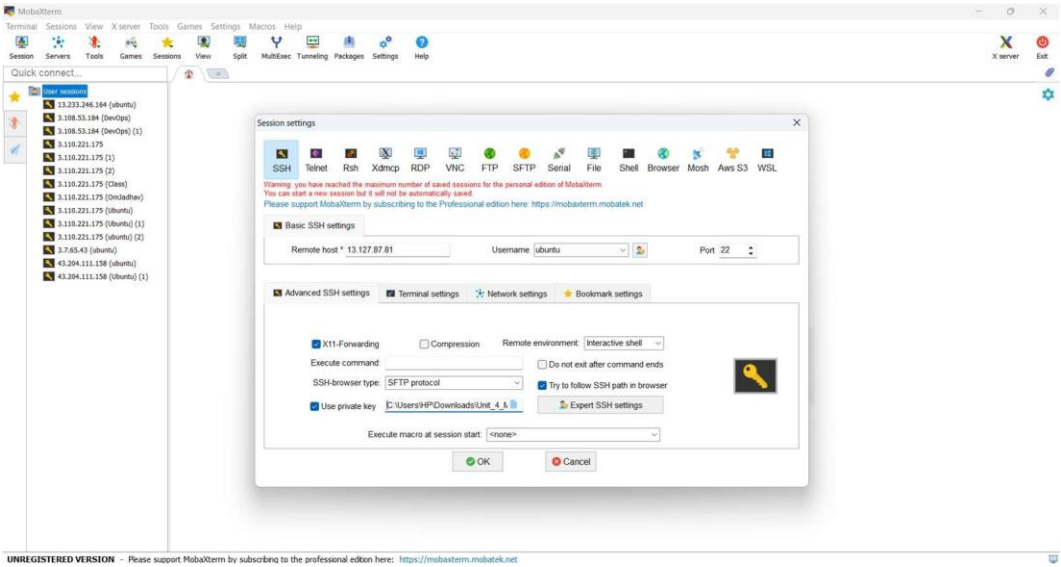
Practical Screenshots / Evidence

Screenshots are arranged in ascending sequence according to their filenames.

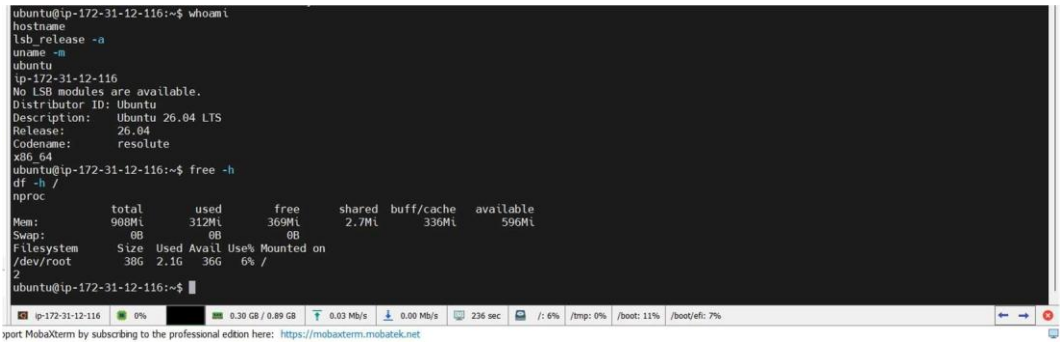
Screenshot 1



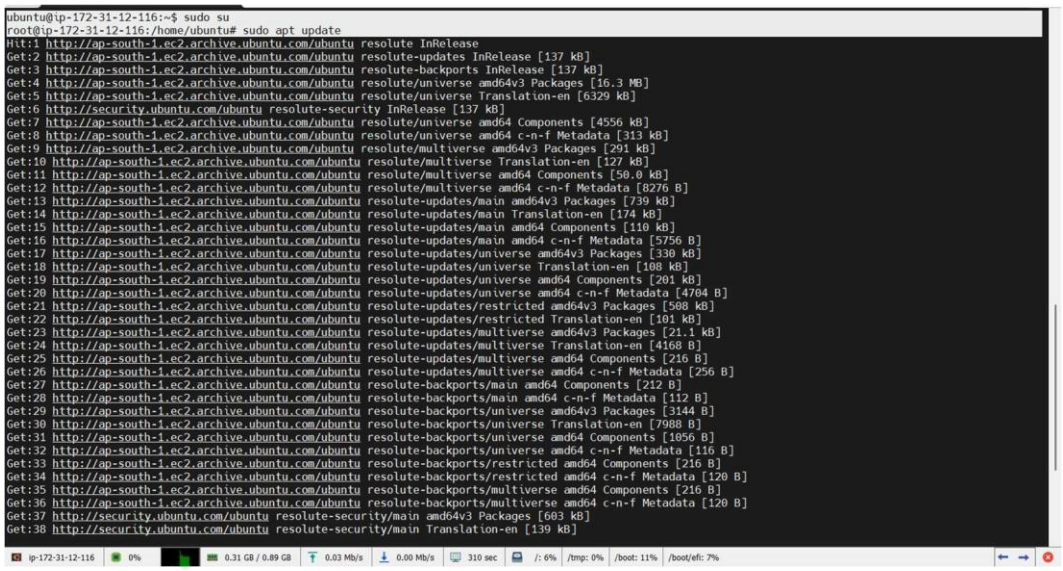
Screenshot 2



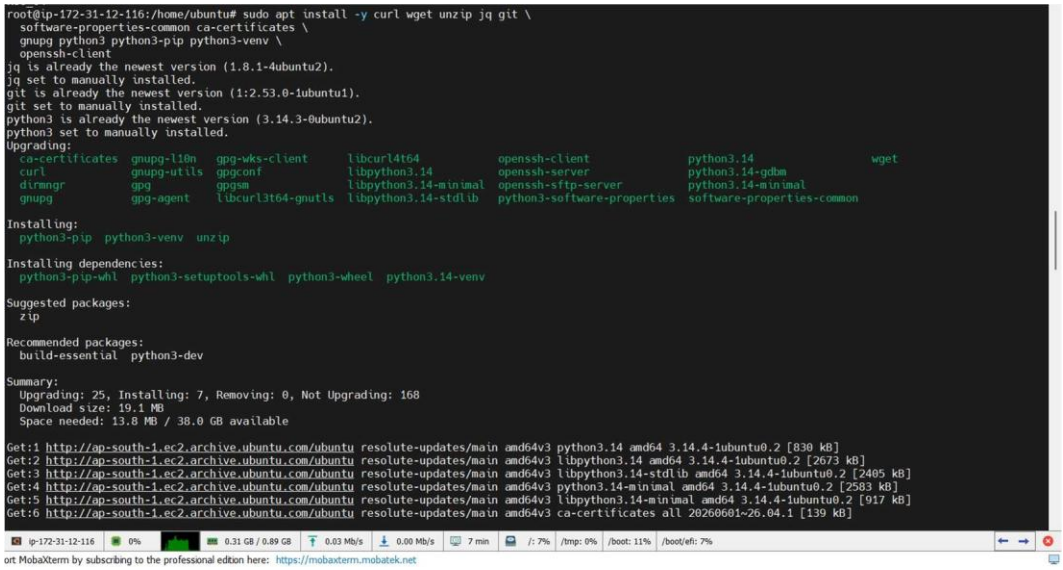
Screenshot 3



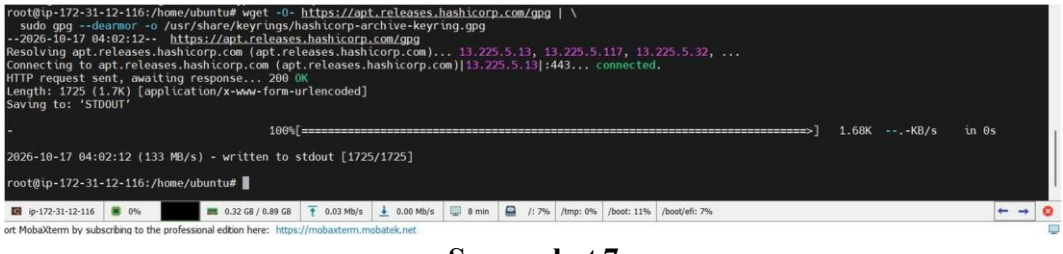
Screenshot 4



Screenshot 5



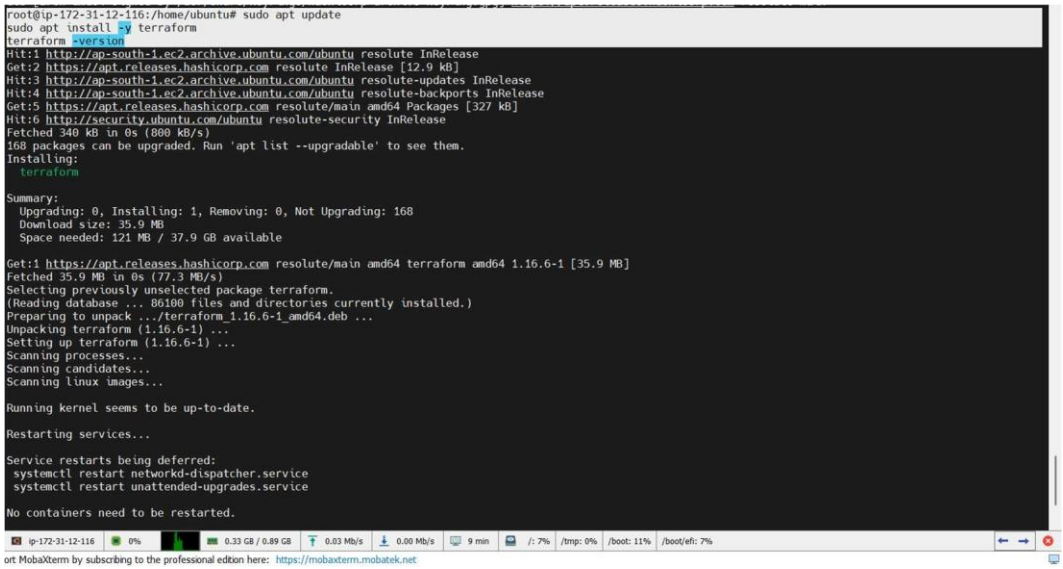
Screenshot 6



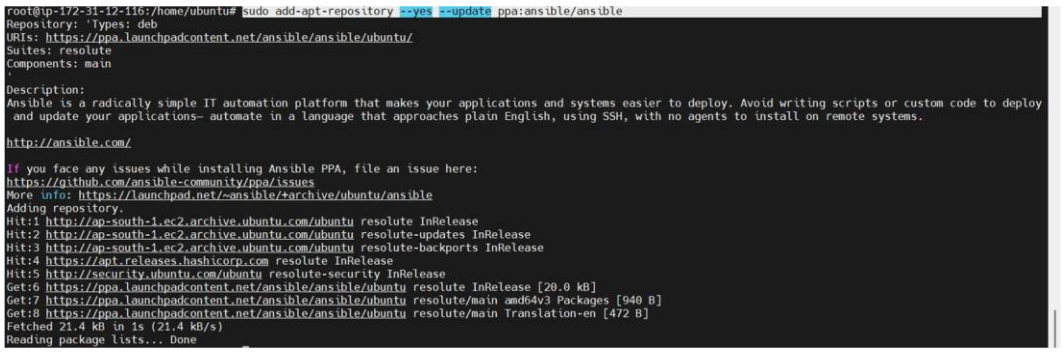
Screenshot 7



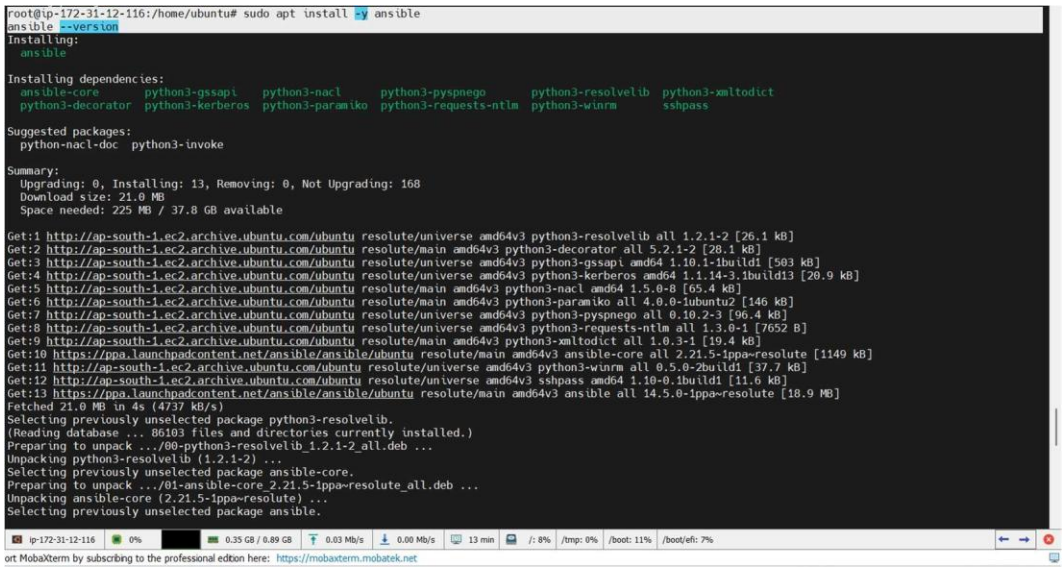
Screenshot 8



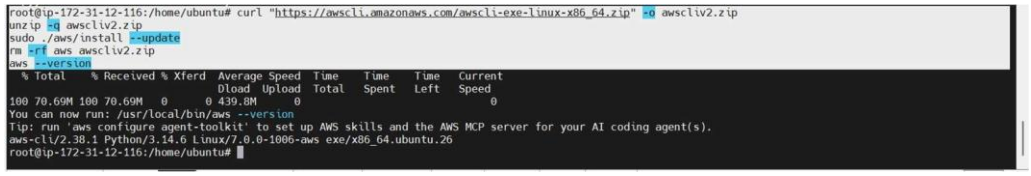
Screenshot 9



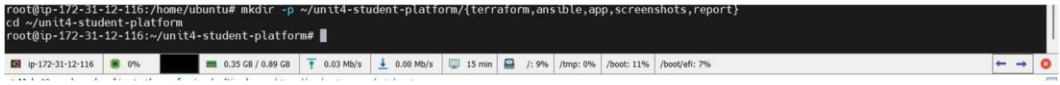
Screenshot 10



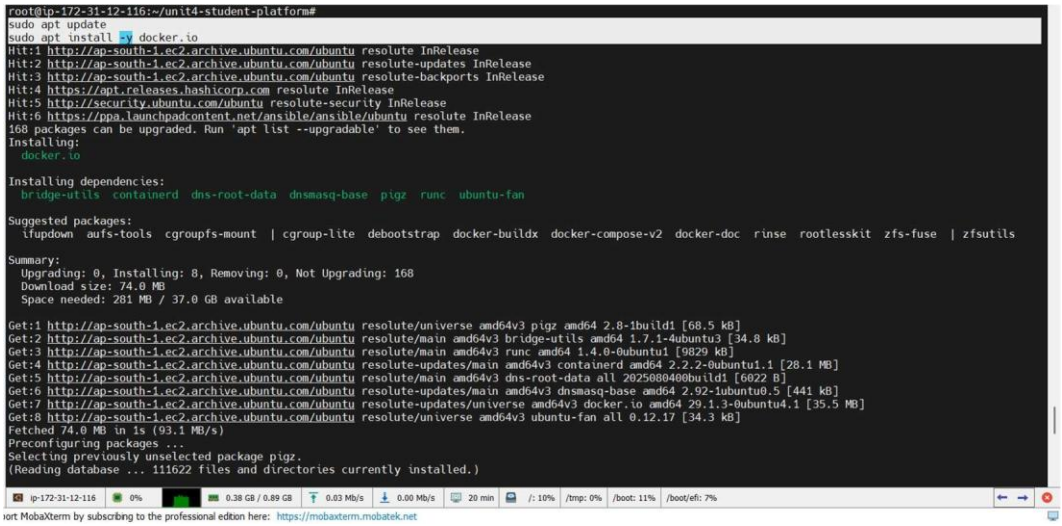
Screenshot 11



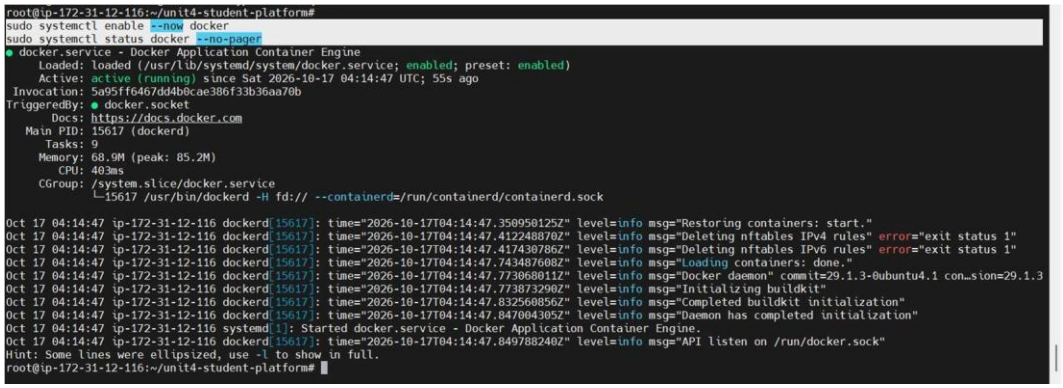
Screenshot 12



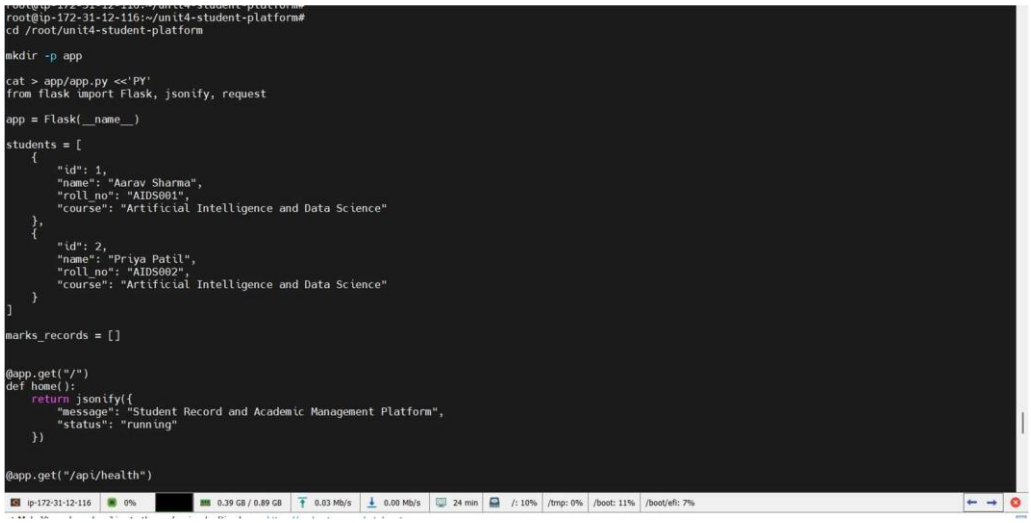
Screenshot 13



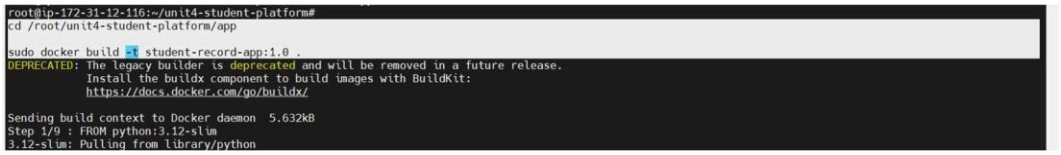
Screenshot 14



Screenshot 15



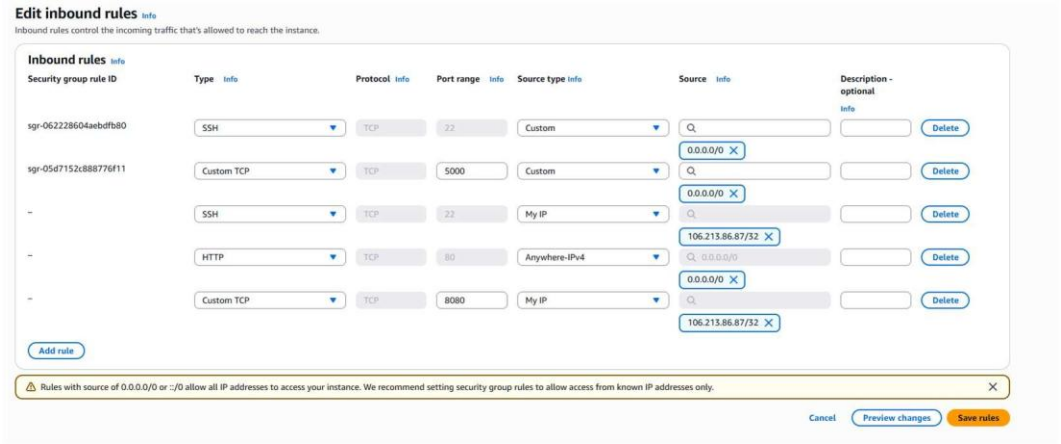
Screenshot 16



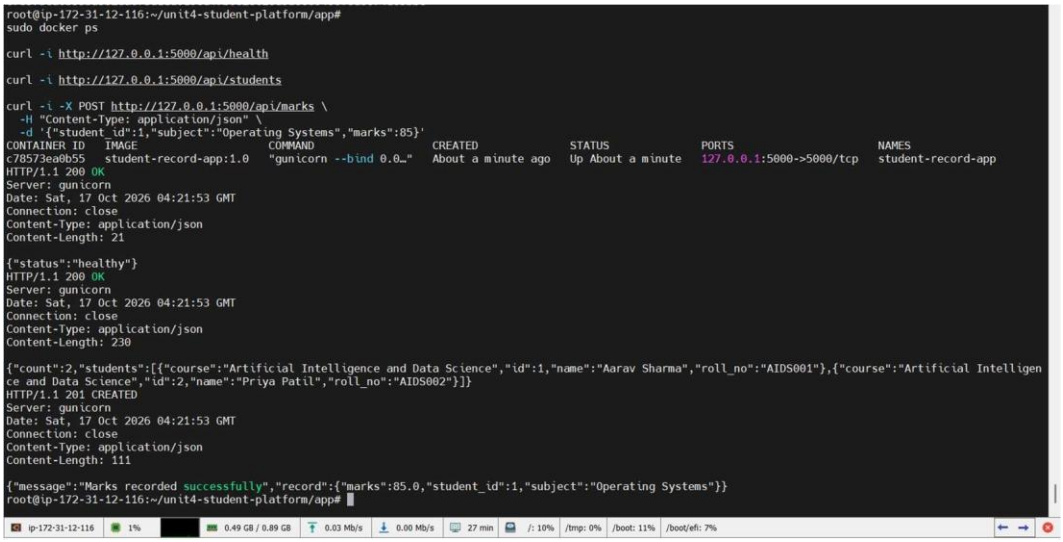
Screenshot 17



Screenshot 18



Screenshot 19



Screenshot 20



Screenshot 21

```
root@ip-172-31-12-116:~/unit4-student-platform/app#
sudo tee /etc/nginx/sites-available/student-platform > /dev/null <<'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

sudo rm -f /etc/nginx/sites-enabled/default
sudo ln -s /etc/nginx/sites-available/student-platform \
  /etc/nginx/sites-enabled/student-platform

sudo nginx -t
sudo systemctl reload nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@ip-172-31-12-116:~/unit4-student-platform/app#
```

Screenshot 22

```
root@ip-172-31-12-116:~/unit4-student-platform/app#
curl -i http://127.0.0.1/api/health
curl -i http://127.0.0.1/api/students
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Sat, 17 Oct 2026 04:24:20 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status":"healthy"}
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Sat, 17 Oct 2026 04:24:20 GMT
Content-Type: application/json
Content-Length: 230
Connection: keep-alive

{"count":2,"students":[{"course":"Artificial Intelligence and Data Science","id":1,"name":"Aarav Sharma","roll_no":"AIDS001"},{"course":"Artificial Intelligen
ce and Data Science","id":2,"name":"Priya Patil","roll_no":"AIDS002"}]}
root@ip-172-31-12-116:~/unit4-student-platform/app#
```

Screenshot 23



Screenshot 24

```
root@ip-172-31-12-116:~/unit4-student-platform/app# sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@ip-172-31-12-116:~/unit4-student-platform/app# curl -i http://127.0.0.1/api/health
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Sat, 17 Oct 2026 04:28:52 GMT
Content-Type: application/json
Content-Length: 21
Connection: keep-alive

{"status":"healthy"}
root@ip-172-31-12-116:~/unit4-student-platform/app#
```

Screenshot 25

```
root@ip-172-31-12-116:~/unit4-student-platform/app#
cd /root/unit4-student-platform

mkdir -p ansible/inventory

mkdir -p ansible/roles/common/tasks
mkdir -p ansible/roles/docker/tasks
mkdir -p ansible/roles/nginx/tasks
mkdir -p ansible/roles/nginx/templates
mkdir -p ansible/roles/library_app/tasks
mkdir -p ansible/roles/library_app/files/app

mkdir -p terraform
mkdir -p report screenshots
root@ip-172-31-12-116:~/unit4-student-platform#
```

Screenshot 26

```
root@ip-172-31-12-116:~/unit4-student-platform#
cat > ansible/inventory/hosts.ini <<'EOF'
[student_servers]
localhost ansible_connection=local
EOF
root@ip-172-31-12-116:~/unit4-student-platform#
```

ip-172-31-12-116 | 0% | 0.44 GB / 0.89 GB | 0.03 Mb/s | 0.00 Mb/s | 35 min | /: 10% | /tmp: 0% | /boot: 11%

Screenshot 27

```
root@ip-172-31-12-116:~/unit4-student-platform#
cat > ansible/roles/common/tasks/main.yml <<'EOF'
---
- name: Update APT package index
  ansible.builtin.apt:
    update_cache: true
    cache_valid_time: 3600

- name: Install common packages
  ansible.builtin.apt:
    name:
      - curl
      - ca-certificates
      - python3
    state: present
EOF
```

ip-172-31-12-116 | 1% | 0.44 GB / 0.89 GB | 0.03 Mb/s | 0.00 Mb/s | 36 min | /: 10% | /tmp: 0% | /boot: 11% | /boot/efi: 7%

Screenshot 28

```
root@ip-172-31-12-116:~/unit4-student-platform#
cat > ansible/roles/docker/tasks/main.yml <<'EOF'
---
- name: Install Docker
  ansible.builtin.apt:
    name: docker.io
    state: present
    update_cache: true

- name: Enable and start Docker
  ansible.builtin.systemd:
    name: docker
    enabled: true
    state: started
EOF
```

ip-172-31-12-116 | 0% | 0.44 GB / 0.89 GB | 0.03 Mb/s | 0.00 Mb/s | 36 min | /: 10% | /tmp: 0% | /boot: 11% | /boot/efi: 7%

Screenshot 29

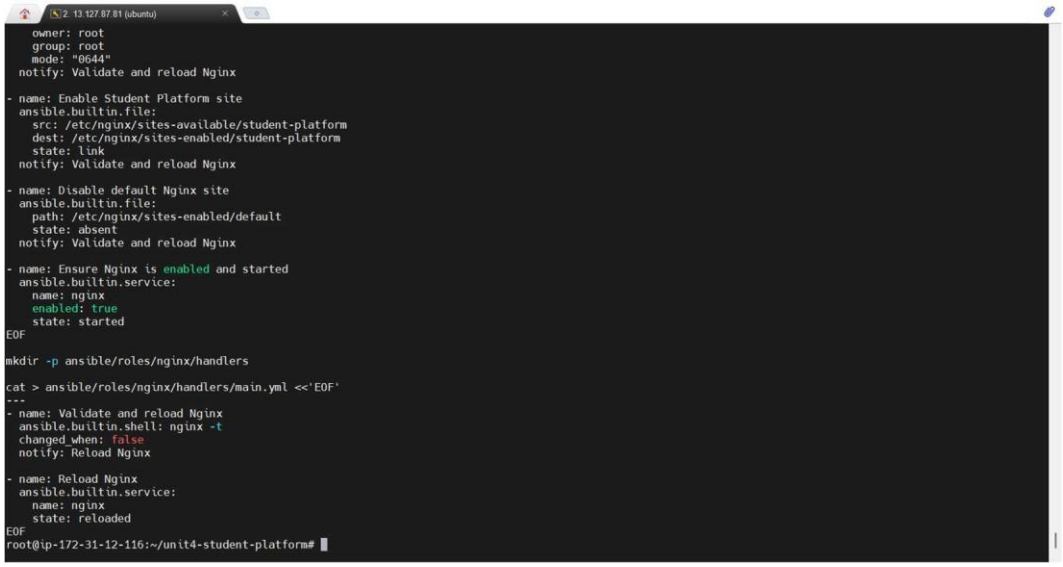
```
root@ip-172-31-12-116:~/unit4-student-platform#
cat > ansible/roles/nginx/templates/student-platform.conf.j2 <<'EOF'
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
root@ip-172-31-12-116:~/unit4-student-platform#
```

Screenshot 30



```
    owner: root
    group: root
    mode: "0644"
  notify: Validate and reload Nginx

- name: Enable Student Platform site
  ansible.builtin.file:
    src: /etc/nginx/sites-available/student-platform
    dest: /etc/nginx/sites-enabled/student-platform
    state: link
  notify: Validate and reload Nginx

- name: Disable default Nginx site
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Validate and reload Nginx

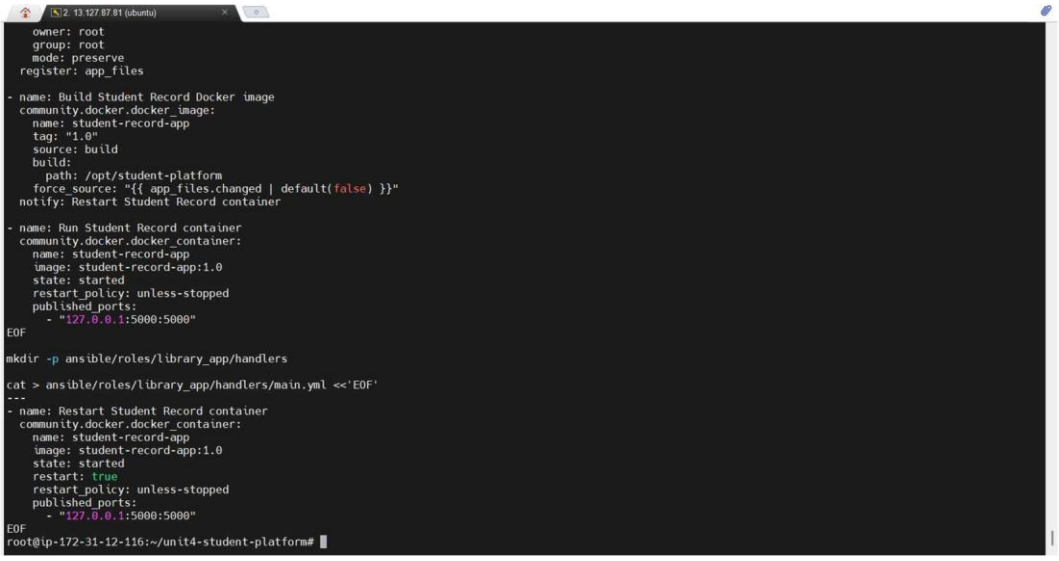
- name: Ensure Nginx is enabled and started
  ansible.builtin.service:
    name: nginx
    enabled: true
    state: started
EOF

mkdir -p ansible/roles/nginx/handlers

cat > ansible/roles/nginx/handlers/main.yml <<'EOF'
---
- name: Validate and reload Nginx
  ansible.builtin.shell: nginx -t
  changed_when: false
  notify: Reload Nginx

- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
EOF
root@ip-172-31-12-116:~/unit4-student-platform#
```

Screenshot 31



```
    owner: root
    group: root
    mode: preserve
  register: app_files

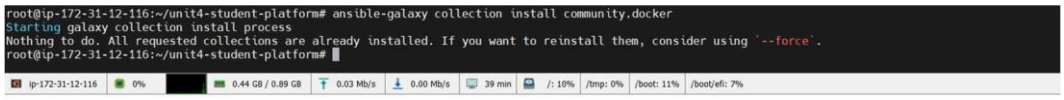
- name: Build Student Record Docker image
  community.docker.docker_image:
    name: student-record-app
    tag: "1.0"
    source: build
    build:
      path: /opt/student-platform
    force_source: "{{ app_files.changed | default(false) }}"
  notify: Restart Student Record container

- name: Run Student Record container
  community.docker.docker_container:
    name: student-record-app
    image: student-record-app:1.0
    state: started
    restart_policy: unless-stopped
    published_ports:
      - "127.0.0.1:5000:5000"
EOF

mkdir -p ansible/roles/library_app/handlers

cat > ansible/roles/library_app/handlers/main.yml <<'EOF'
---
- name: Restart Student Record container
  community.docker.docker_container:
    name: student-record-app
    image: student-record-app:1.0
    state: started
    restart: true
    restart_policy: unless-stopped
    published_ports:
      - "127.0.0.1:5000:5000"
EOF
root@ip-172-31-12-116:~/unit4-student-platform#
```

Screenshot 32



```
root@ip-172-31-12-116:~/unit4-student-platform# ansible-galaxy collection install community.docker
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using `--force`.
root@ip-172-31-12-116:~/unit4-student-platform#
```

ip-172-31-12-116 | 0% | 0.44 GB / 0.89 GB | 0.03 Mb/s | 0.00 Mb/s | 39 min | /: 10% | /tmp: 0% | /boot: 11% | /boot/efi: 7%

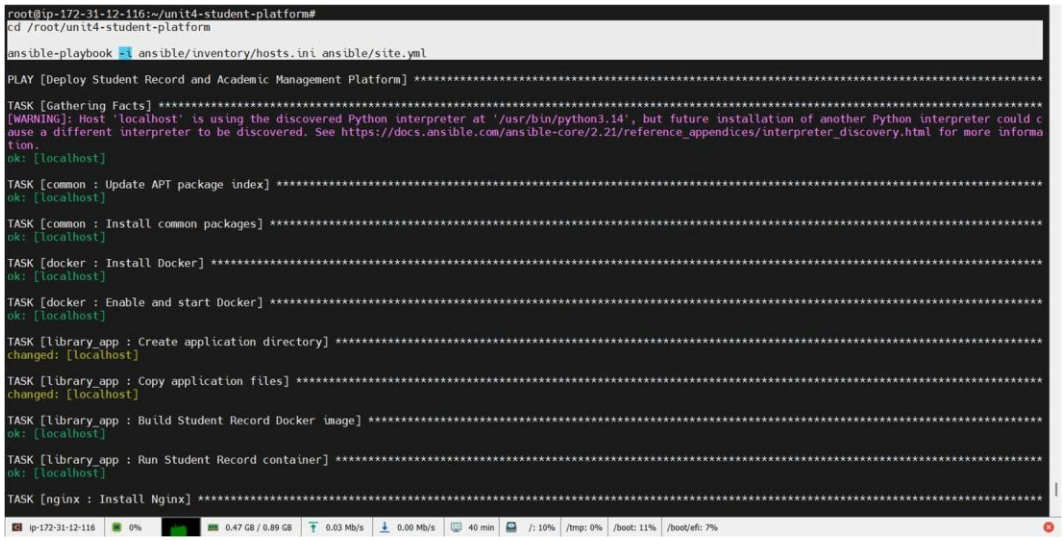
Screenshot 33



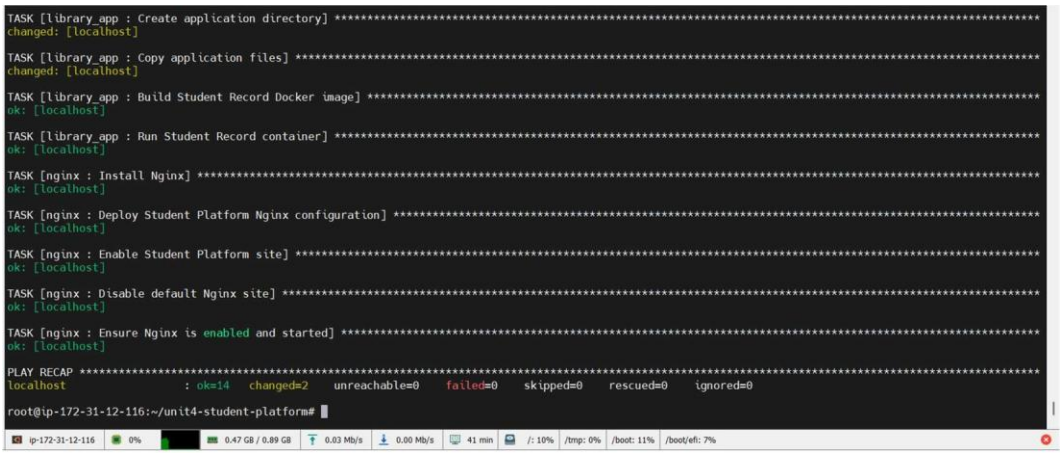
```
root@ip-172-31-12-116:~/unit4-student-platform#
cat > ansible/site.yml <<'EOF'
---
- name: Deploy Student Record and Academic Management Platform
  hosts: student_servers
  become: true

  roles:
    - common
    - docker
    - library_app
    - nginx
EOF
```

Screenshot 34



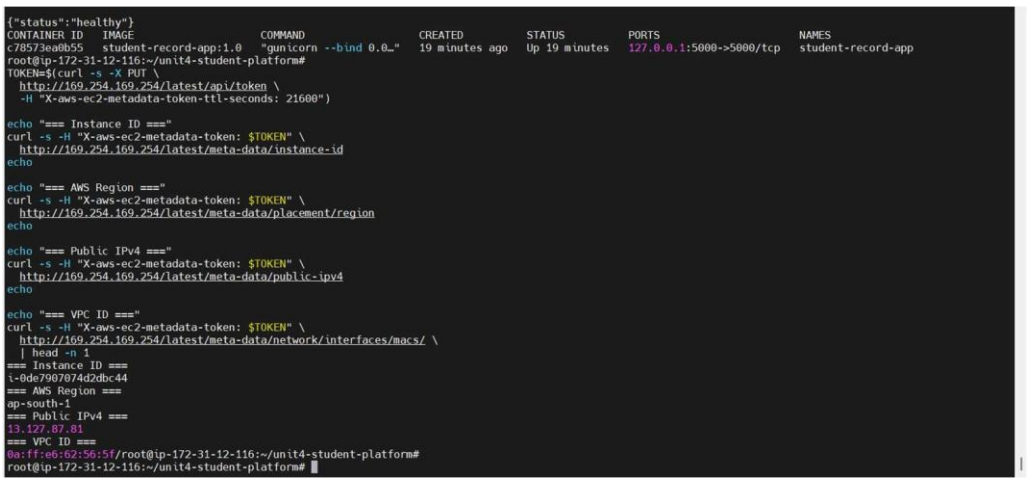
Screenshot 35



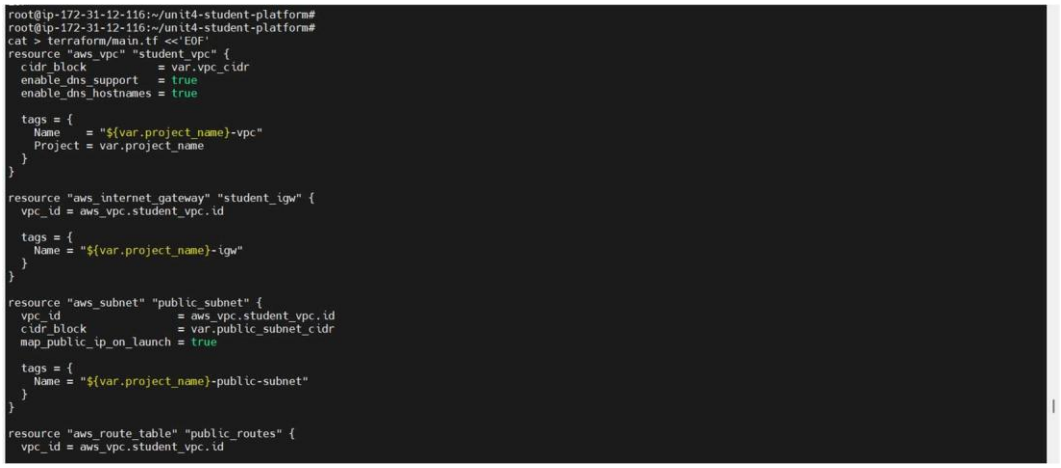
Screenshot 36



Screenshot 37



Screenshot 38



```
root@ip-172-31-12-116:~/unit4-student-platform#
root@ip-172-31-12-116:~/unit4-student-platform#
cat > terraform/main.tf <<'EOF'
resource "aws_vpc" "student_vpc" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name    = "${var.project_name}-vpc"
    Project = var.project_name
  }
}

resource "aws_internet_gateway" "student_igw" {
  vpc_id = aws_vpc.student_vpc.id

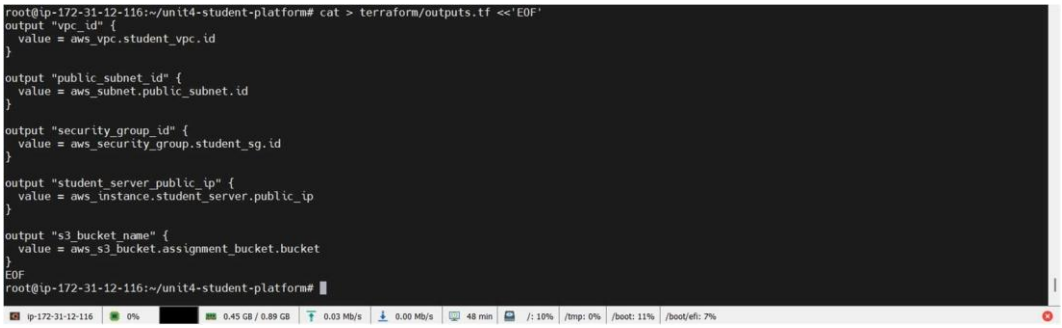
  tags = {
    Name = "${var.project_name}-igw"
  }
}

resource "aws_subnet" "public_subnet" {
  vpc_id                  = aws_vpc.student_vpc.id
  cidr_block              = var.public_subnet_cidr
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-subnet"
  }
}

resource "aws_route_table" "public_routes" {
  vpc_id = aws_vpc.student_vpc.id
```

Screenshot 39



```
root@ip-172-31-12-116:~/unit4-student-platform# cat > terraform/outputs.tf <<'EOF'
output "vpc_id" {
  value = aws_vpc.student_vpc.id
}

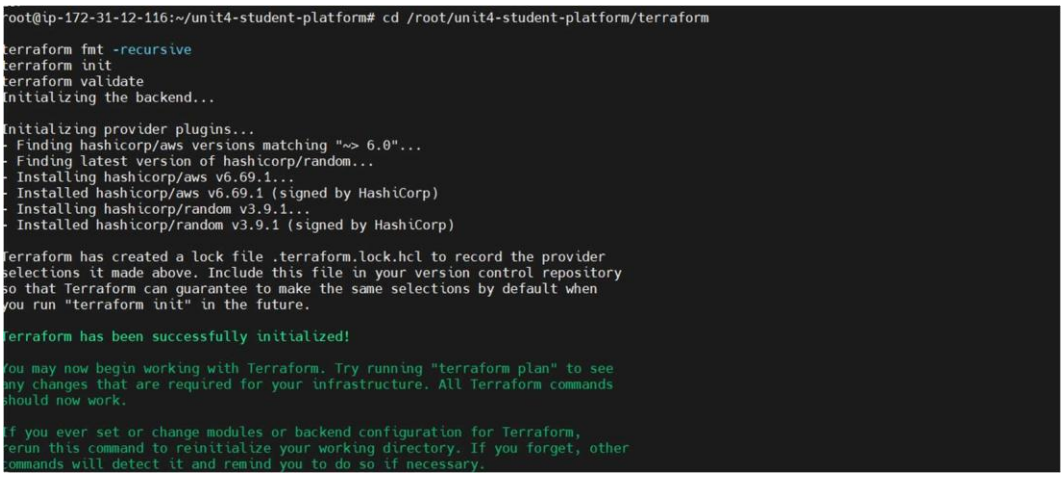
output "public_subnet_id" {
  value = aws_subnet.public_subnet.id
}

output "security_group_id" {
  value = aws_security_group.student_sg.id
}

output "student_server_public_ip" {
  value = aws_instance.student_server.public_ip
}

output "s3_bucket_name" {
  value = aws_s3_bucket.assignment_bucket.bucket
}
EOF
root@ip-172-31-12-116:~/unit4-student-platform#
ip-172-31-12-116  0%  0.45 GB / 0.89 GB  0.03 Mb/s  0.00 Mb/s  48 min  /: 10%  /tmp: 0%  /boot: 11%  /boot/efi: 7%
```

Screenshot 40



```
root@ip-172-31-12-116:~/unit4-student-platform# cd /root/unit4-student-platform/terraform

terraform fmt -recursive
terraform init
terraform validate
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/aws v6.69.1...
- Installed hashicorp/aws v6.69.1 (signed by HashiCorp)
- Installing hashicorp/random v3.9.1...
- Installed hashicorp/random v3.9.1 (signed by HashiCorp)

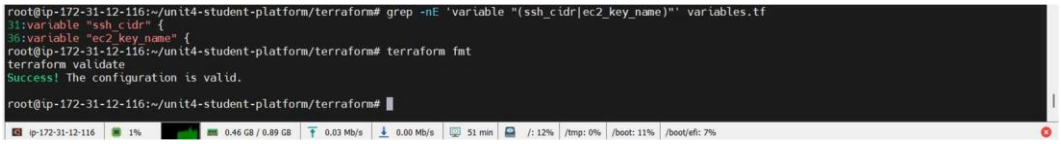
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Screenshot 41



```
root@ip-172-31-12-116:~/unit4-student-platform/terraform# grep -nE 'variable "(ssh_cidr|ec2_key_name)"' variables.tf
31:variable "ssh_cidr" {
36:variable "ec2_key_name" {
root@ip-172-31-12-116:~/unit4-student-platform/terraform# terraform fmt
terraform validate
Success! The configuration is valid.

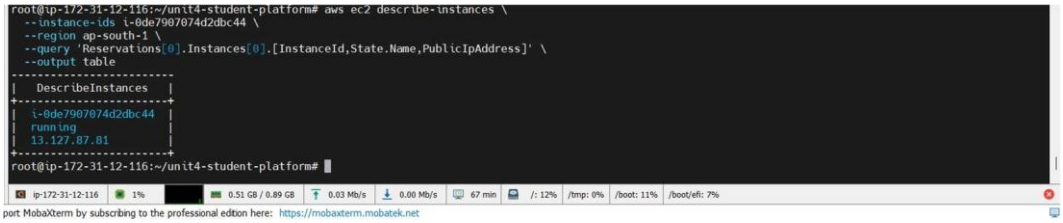
root@ip-172-31-12-116:~/unit4-student-platform/terraform#
ip-172-31-12-116  1%  0.46 GB / 0.89 GB  0.03 Mb/s  0.00 Mb/s  51 min  /: 12%  /tmp: 0%  /boot: 11%  /boot/efi: 7%
```

Screenshot 42

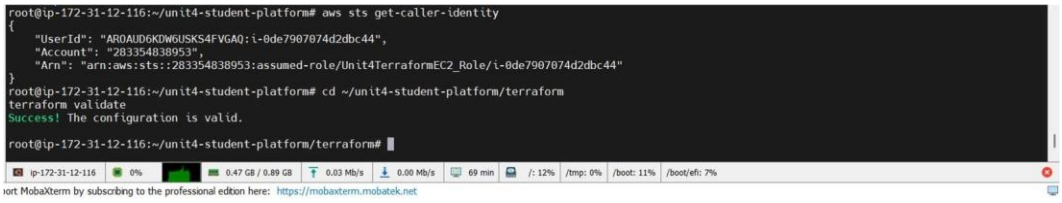


```
root@ip-172-31-12-116:~/unit4-student-platform# aws sts get-caller-identity
{
    "UserId": "AROAUD6KDW6USKS4FVGAQ:i-0de7907074d2dbc44",
    "Account": "283354838953",
    "Arn": "arn:aws:sts::283354838953:assumed-role/Unit4TerraformEC2_Role/i-0de7907074d2dbc44"
}
root@ip-172-31-12-116:~/unit4-student-platform#
```

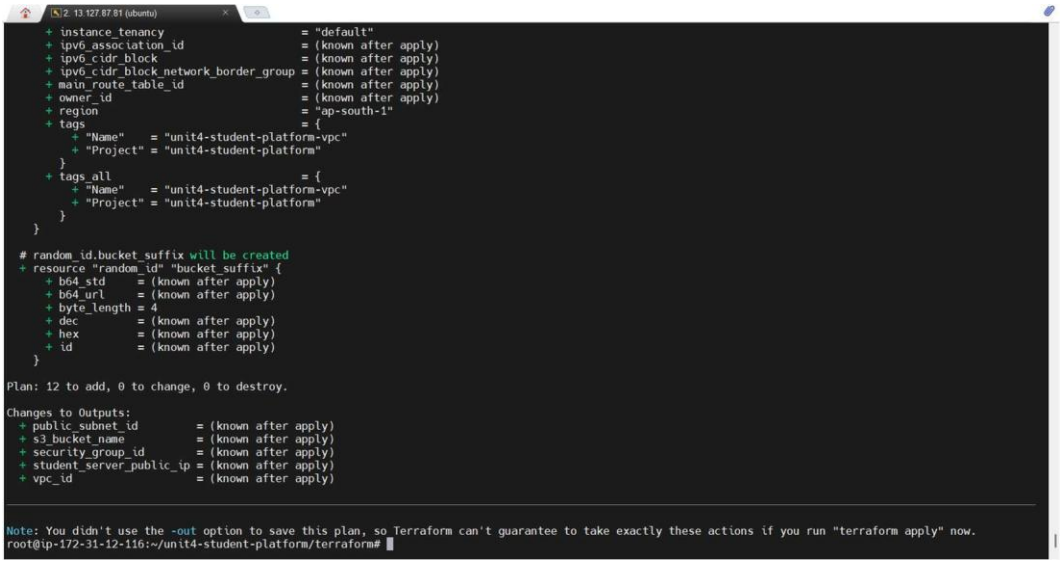
Screenshot 43



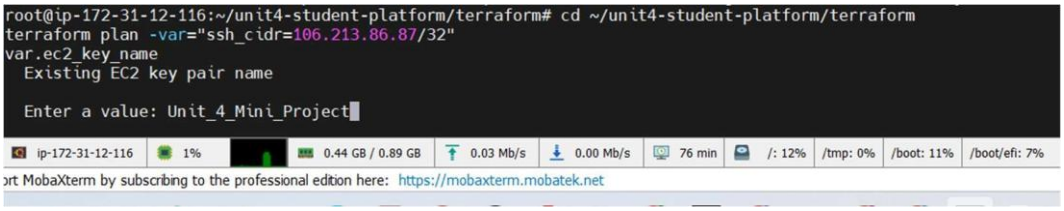
Screenshot 44



Screenshot 45



Screenshot 46



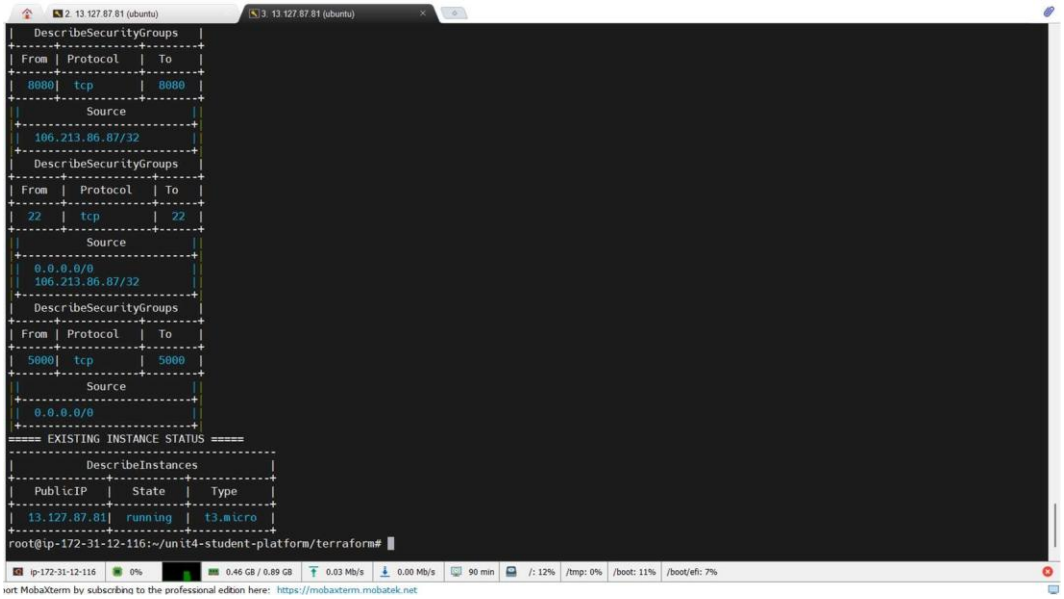
Screenshot 47



Screenshot 48

```
root@ip-172-31-12-116:~/unit4-student-platform/terraform# aws ec2 describe-instances \
  --instance-ids i-0de7907074d2dbc44 \
  --region ap-south-1 \
  --query 'Reservations[0].Instances[0].{VPC:VpcId,Subnet:SubnetId,SecurityGroups:SecurityGroups[*].GroupId,KeyName:KeyName,PublicIP:PublicIpAddress,Type:Inst
anceType}' \
  --output json
{
    "VPC": "vpc-04e67ea523a673c72",
    "Subnet": "subnet-00aad1150514edf09",
    "SecurityGroups": [
        "sg-08ceebe36f6ee31c2"
    ],
    "KeyName": "Unit_4_Mini_Project",
    "PublicIP": "13.127.87.81",
    "Type": "t3.micro"
}
root@ip-172-31-12-116:~/unit4-student-platform/terraform# aws ec2 describe-instances \
  --instance-ids i-0de7907074d2dbc44 \
  --region ap-south-1 \
  --query 'Reservations[0].Instances[0].[VpcId,SubnetId]' \
  --output table
------------------------------
|      DescribeInstances     |
+----------------------------+
|  vpc-04e67ea523a673c72     |
|  subnet-00aad1150514edf09  |
+----------------------------+
root@ip-172-31-12-116:~/unit4-student-platform/terraform# aws s3api list-buckets \
  --query 'Buckets[].Name' \
  --output table
root@ip-172-31-12-116:~/unit4-student-platform/terraform# ^C
root@ip-172-31-12-116:~/unit4-student-platform/terraform# aws ec2 describe-vpcs \
  --vpc-ids vpc-04e67ea523a673c72 \
  --region ap-south-1 \
  --query 'Vpcs[0].{CIDR:CidrBlock,State:State,IsDefault:IsDefault}' \
```

Screenshot 49



Screenshot 50

```
output "s3_bucket_name" {
  value = aws_s3_bucket.assignment_bucket.bucket
}
EOF
root@ip-172-31-12-116:~/unit4-student-platform/terraform# terraform fmt
terraform init
terraform validate
terraform plan
Initializing the backend...

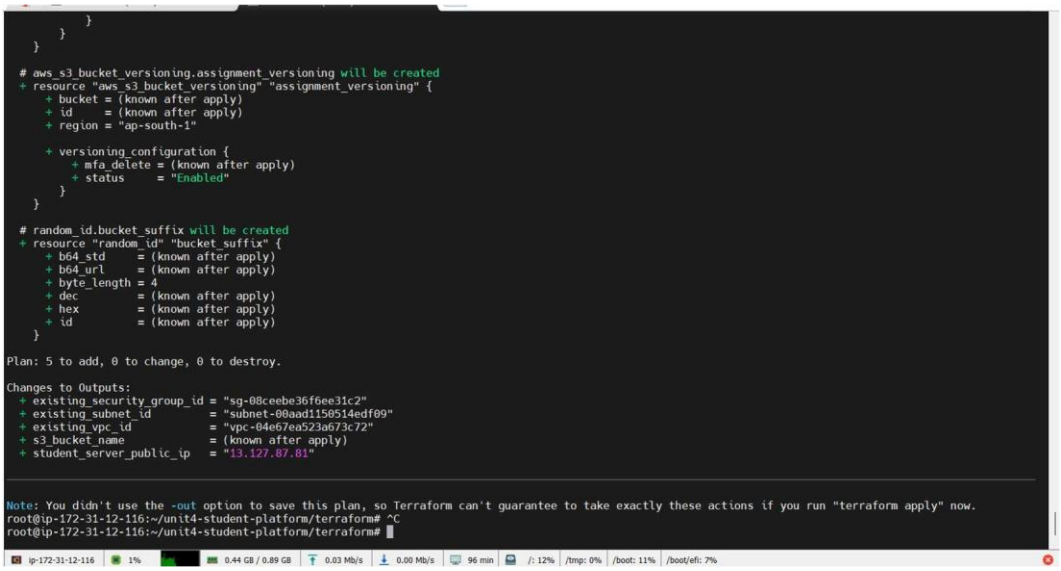
Initializing provider plugins...
- Reusing previous version of hashicorp/random from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/random v3.9.1
- Using previously-installed hashicorp/aws v6.69.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Screenshot 51



```
        }
      }
    }

  # aws_s3_bucket_versioning.assignment_versioning will be created
  + resource "aws_s3_bucket_versioning" "assignment_versioning" {
      + bucket = (known after apply)
      + id     = (known after apply)
      + region = "ap-south-1"

      + versioning_configuration {
          + mfa_delete = (known after apply)
          + status     = "Enabled"
        }
    }

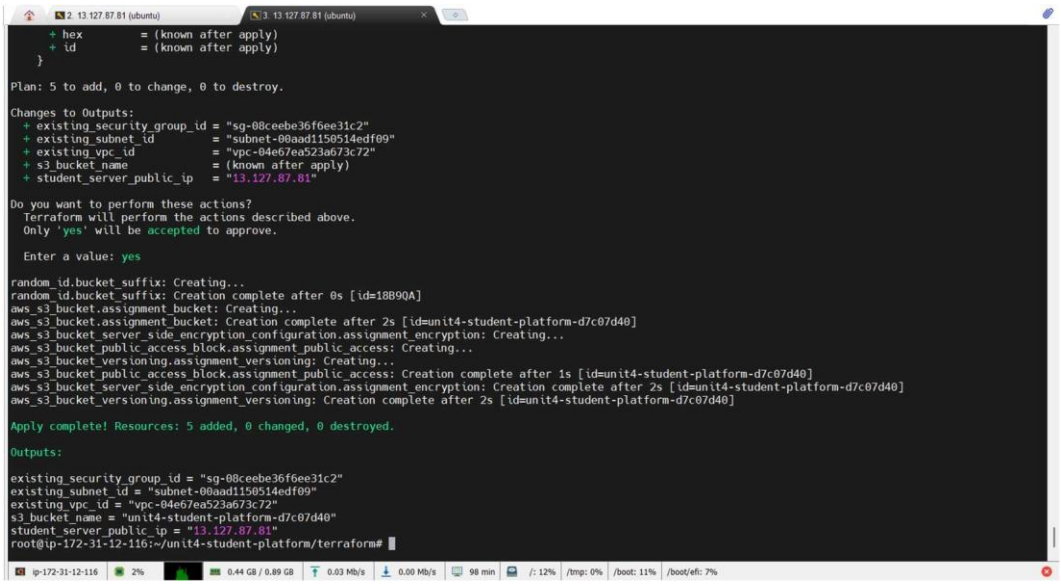
  # random_id.bucket_suffix will be created
  + resource "random_id" "bucket_suffix" {
      + b64_std     = (known after apply)
      + b64_url     = (known after apply)
      + byte_length = 4
      + dec         = (known after apply)
      + hex         = (known after apply)
      + id          = (known after apply)
    }

Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + existing_security_group_id = "sg-08ceebe36f6ee31c2"
  + existing_subnet_id         = "subnet-00aad1150514edf09"
  + existing_vpc_id            = "vpc-04e67ea523a673c72"
  + s3_bucket_name             = (known after apply)
  + student_server_public_ip   = "13.127.87.81"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
root@ip-172-31-12-116:~/unit4-student-platform/terraform# ^C
root@ip-172-31-12-116:~/unit4-student-platform/terraform#
```


Screenshot 52



```
      + hex         = (known after apply)
      + id          = (known after apply)
    }

Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + existing_security_group_id = "sg-08ceebe36f6ee31c2"
  + existing_subnet_id         = "subnet-00aad1150514edf09"
  + existing_vpc_id            = "vpc-04e67ea523a673c72"
  + s3_bucket_name             = (known after apply)
  + student_server_public_ip   = "13.127.87.81"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

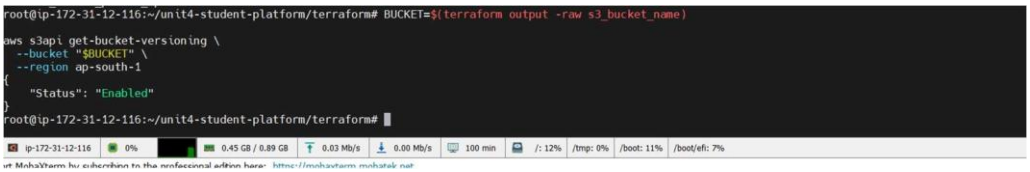
random_id.bucket_suffix: Creating...
random_id.bucket_suffix: Creation complete after 0s [id=18B9QA]
aws_s3_bucket.assignment_bucket: Creating...
aws_s3_bucket.assignment_bucket: Creation complete after 2s [id=unit4-student-platform-d7c07d40]
aws_s3_bucket_server_side_encryption_configuration.assignment_encryption: Creating...
aws_s3_bucket_public_access_block.assignment_public_access: Creating...
aws_s3_bucket_versioning.assignment_versioning: Creating...
aws_s3_bucket_public_access_block.assignment_public_access: Creation complete after 1s [id=unit4-student-platform-d7c07d40]
aws_s3_bucket_server_side_encryption_configuration.assignment_encryption: Creation complete after 2s [id=unit4-student-platform-d7c07d40]
aws_s3_bucket_versioning.assignment_versioning: Creation complete after 2s [id=unit4-student-platform-d7c07d40]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

existing_security_group_id = "sg-08ceebe36f6ee31c2"
existing_subnet_id = "subnet-00aad1150514edf09"
existing_vpc_id = "vpc-04e67ea523a673c72"
s3_bucket_name = "unit4-student-platform-d7c07d40"
student_server_public_ip = "13.127.87.81"
root@ip-172-31-12-116:~/unit4-student-platform/terraform#
```

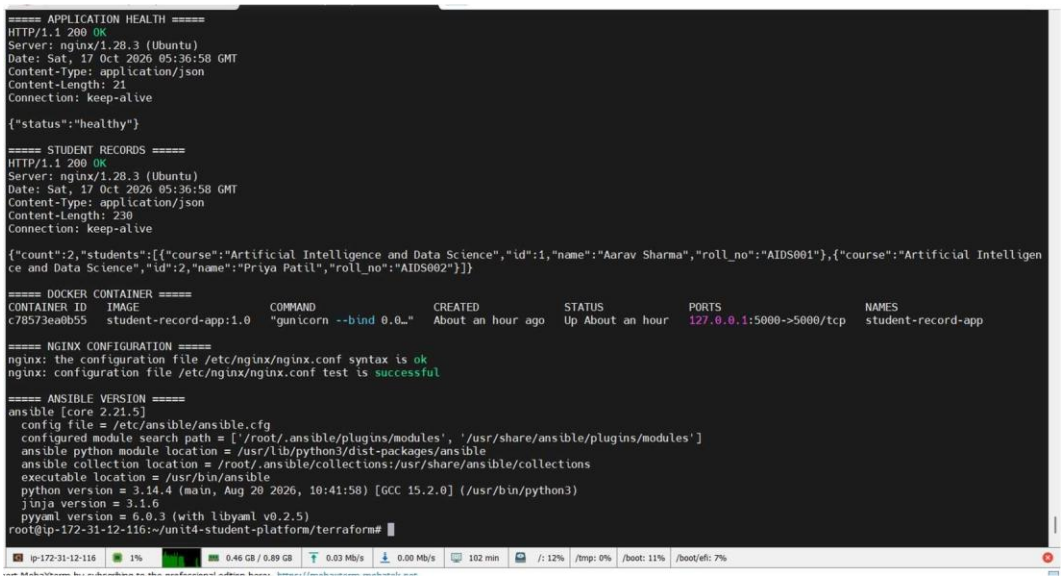

Screenshot 53



```
root@ip-172-31-12-116:~/unit4-student-platform/terraform# BUCKET=$(terraform output -raw s3_bucket_name)

aws s3api get-bucket-versioning \
  --bucket "$BUCKET" \
  --region ap-south-1
{
    "Status": "Enabled"
}
root@ip-172-31-12-116:~/unit4-student-platform/terraform#
```

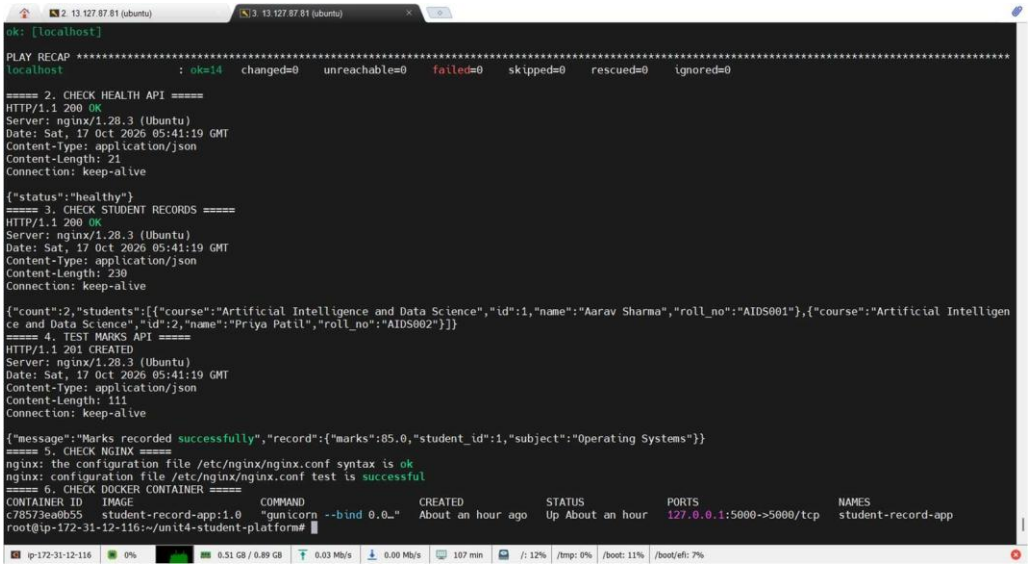
Screenshot 54



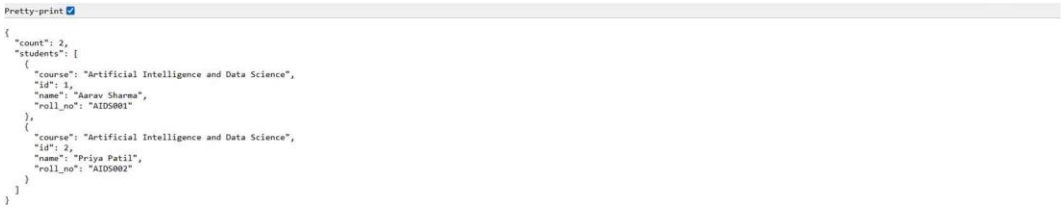
Screenshot 55



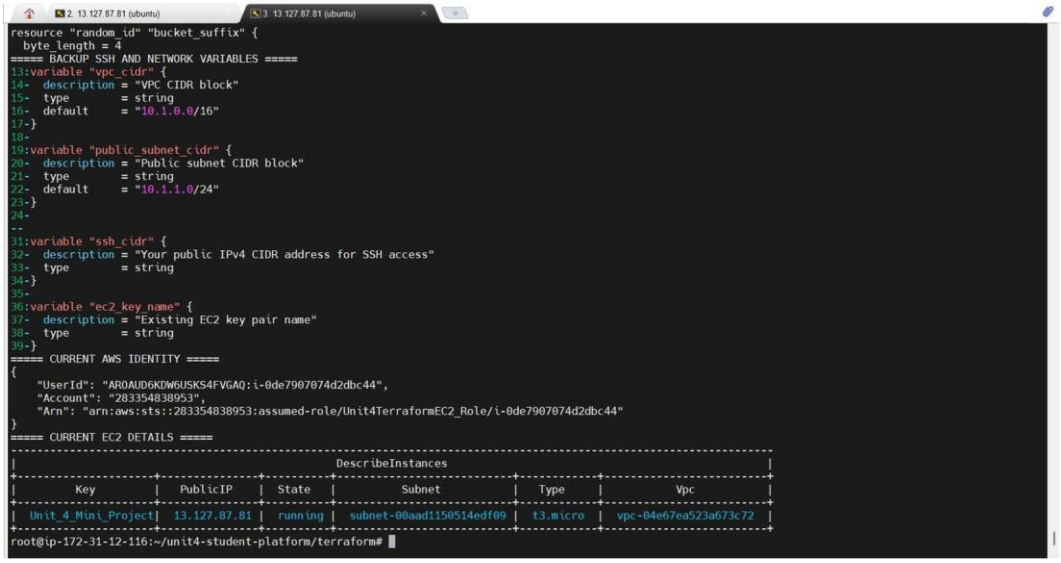
Screenshot 56



Screenshot 57



Screenshot 58



Screenshot 59



Screenshot 60

